

SOURCE CODE EXTRACT

Software Title: Secure Web-Based Online Compiler for C++ and Java Using Docker-Based Containerized Execution

Author: Appurav Nehra

Year of Creation: 2026

FILE: backend/server.js

```
const express = require('express');
const { exec, execSync } = require('child_process');
const fs = require('fs').promises;
const path = require('path');
const cors = require('cors');
const util = require('util');
const app = express();
app.use(express.json());
app.use(cors());
const log = (...args) => {
  console.log('[ ' + new Date().toISOString() + ']', ...args);
  process.stdout.write("");
};
const error = (...args) => {
  console.error('[ ' + new Date().toISOString() + ']', ...args);
  process.stderr.write("");
};
const TEMP_DIR = path.join(__dirname, 'temp');
1
const execPromise = util.promisify(exec);
(async () => {
  try {
    log('Starting server initialization...');
    await fs.mkdir(TEMP_DIR, { recursive: true });
    log(`Created/verified temp directory at: ${TEMP_DIR}`);
    log('Checking Docker image...');
    const dockerResult = execSync('docker image inspect code-execution-env || docker build -t code-execution-env .', { encoding: 'utf8' });
    log('Docker image is ready');
  } catch (err) {
    error('Initialization error:', err);
  }
})();
async function cleanupFiles(filenamees) {
  for (const filename of filenamees) {
```

```

try {
  await fs.unlink(filename);
  log(`Cleaned up file: ${filename}`);
} catch (err) {
  error(`Error cleaning up ${filename}:`, err);
}
}
}

function executeInDocker(language, code, input = "") {
  return new Promise(async (resolve, reject) => {
    const timestamp = Date.now();
    const className = 'Main';
    const codeFile = path.join(TEMP_DIR, `${className}.${language} === 'java' ? 'java' : 'cpp'`);
    const inputFile = path.join(TEMP_DIR, `input_${timestamp}.txt`);
    try {
      log(`Starting execution for ${language}`);
      log("Writing files...");
      await fs.writeFile(codeFile, code);
      await fs.writeFile(inputFile, input);
      log("Files written successfully");
      const dockerCmd = `docker run --rm \
-v "${TEMP_DIR}:/app/code" \
2
--network none \
--memory=512m \
--cpus=1 \
code-execution-env \
${language} "/app/code/${path.basename(codeFile)}"
"/app/code/${path.basename(inputFile)}";
      log('Executing Docker command:', dockerCmd);
      const { stdout, stderr } = await execPromise(dockerCmd, { timeout: 15000 });
      log("Docker execution completed");
      if (stdout) log('stdout:', stdout);
      if (stderr) log('stderr:', stderr);
      await cleanupFiles([codeFile, inputFile]);
      if (stdout || !stderr) {
        resolve({
          status: 'success',
          output: stdout.trim(),
          error: stderr
        });
      } else {
        resolve({

```

```

status: 'error',
message: stderr,
details: { stdout, stderr }
});
}
} catch (err) {
error('Error in executeInDocker:', err);
await cleanupFiles([codeFile, inputFile]);
if (err.killed) {
resolve({ status: 'error', message: 'Execution timed out' });
} else {
resolve({
status: 'error',
message: err.stderr || err.message,
details: { stdout: err.stdout, stderr: err.stderr, error: err.message }
});
}
}
}
3
});
}
app.post('/api/compile', async (req, res) => {
const { language, code, input = '' } = req.body;
log('\n=== New Compilation Request ===');
log('Language:', language);
log('Code:', code);
log('Input:', input);
if (!language || !code) {
error('Missing required fields');
return res.status(400).json({
status: 'error',
message: 'Language and code are required'
});
}
if (!['java', 'cpp'].includes(language)) {
error('Unsupported language:', language);
return res.status(400).json({
status: 'error',
message: 'Unsupported language'
});
}
try {
const result = await executeInDocker(language, code, input);
log('Execution result:', result);

```

```

res.json(result);
} catch (err) {
  error('Error in /api/compile:', err);
  res.status(500).json({
    status: 'error',
    message: err.message || 'Internal server error',
    details: err
  });
}
});
const PORT = 9000;
app.listen(PORT, () => {
  log(`Server running on port ${PORT}`);
  log(`Temp directory: ${TEMP_DIR}`);
  4
  log('Ready to accept requests');
});
app.get('/health', (req, res) => {
  log('Health check request received');
  res.json({ status: 'healthy' });
});

```

FILE: backend/run.sh

```

#!/bin/bash
# Get arguments
LANGUAGE=$1
CODE_FILE=$2
INPUT_FILE=$3
# Function to handle errors
handle_error() {
  echo "Error: $1" >&2
  exit 1
}
case "$LANGUAGE" in
  "cpp")
    # Compile and run C++ code
    g++ -o /app/program "$CODE_FILE" || handle_error "Compilation failed"
    if [ -f "$INPUT_FILE" ]; then
      /app/program < "$INPUT_FILE"
    else
      /app/program
    fi
  ;;

```

```

"java")
# Get the directory containing the source file
DIR=$(dirname "$CODE_FILE")
CLASS_NAME=$(basename "$CODE_FILE" .java)
# Compile Java code
cd "$DIR" || handle_error "Failed to change to source directory"
javac "$CLASS_NAME.java" || handle_error "Compilation failed"
5
# Run Java code
if [ -f "$INPUT_FILE" ]; then
java "$CLASS_NAME" < "$INPUT_FILE"
else
java "$CLASS_NAME"
fi
;;
*)
handle_error "Unsupported language: $LANGUAGE"
;;
esac

```

FILE: my-app/app/page.tsx

```

"use client"
import { useState } from "react";
import MonacoEditor from "@monaco-editor/react";
import axios from "axios";
export default function Home() {
const [code, setCode] = useState("");
const [output, setOutput] = useState("");
const [input, setInput] = useState("");
const [language, setLanguage] = useState("cpp");
const [showInput, setShowInput] = useState(false);
const checkForInputOperations = (code:any) => {
const inputPatterns = {
cpp: /\b(cin|scanf|getline)\b/,
java: /\b(Scanner|BufferedReader|System\.console\(\)\.readLine)\b/
};
return inputPatterns[language].test(code);
};
const handleCodeChange = (value) => {
setCode(value);
};
const handleCompile = async () => {
try{

```

```

const needsInput = checkForInputOperations(code);
if (needsInput && !showInput) {
  setShowInput(true);
  6
  return;
}
if (needsInput && !input.trim()) {
  setOutput("Please provide input first");
  return;
}
console.log("Sending request with:", {
  language,
  code,
  input: needsInput ? input : ""
});
const response = await axios.post("http://localhost:9000/api/compile", {
  language,
  code,
  input: needsInput ? input : ""
});
console.log("Response received:", response.data);
if (response.data.output !== undefined) {
  setOutput(response.data.output);
  setShowInput(false);
} else {
  setOutput("No output received from server");
}
} catch (error) {
  console.error("Error:", error);
  setOutput("Error: " + (error.response?.data?.message || error.message));
}
};
return (
  <div className="flex flex-col items-center justify-center min-h-screen bg-gray-900 p-6">
    <h1 className="text-3xl font-semibold text-white mb-6">Online Code Compiler</h1>
    <div className="flex flex-col md:flex-row items-center gap-4 mb-6">
      <label htmlFor="language" className="text-gray-300 font-medium">
        Choose Language:
      </label>
      <select
        id="language"
        7
        onChange={(e) => setLanguage(e.target.value)}
        value={language}

```

```

className="px-4 py-2 bg-gray-700 text-gray-200 rounded-md focus:outline-none"
>
<option value="cpp">C++</option>
<option value="java">Java</option>
</select>
</div>
<div className="w-full max-w-4xl mb-6">
<div className="rounded-lg overflow-hidden shadow-lg">
<MonacoEditor
height="400px"
language={language === "cpp" ? "cpp" : "java"}
theme="vs-dark"
value={code}
onChange={handleCodeChange}
options={{
minimap: { enabled: false },
fontSize: 14,
}}
/>
</div>
{}
{showInput && (
<div className="mt-4 bg-gray-800 rounded-lg p-4">
<label className="block text-gray-300 font-medium mb-2">
Program Input:
</label>
<textarea
value={input}
onChange={(e) => setInput(e.target.value)}
placeholder={`Enter input values here${language === 'cpp' ? ' (for cin)' : ' (for Scanner)'} `}
className="w-full h-32 px-3 py-2 bg-gray-700 text-gray-200 rounded-md
focus:outline-none resize-none"
/>
<p className="text-gray-400 text-sm mt-2">
{language === 'cpp'
? 'For multiple inputs, separate values with spaces or newlines'
: 'For multiple inputs, separate values with spaces or newlines'}
</p>
</div>
8
)}
</div>
<button

```

```
onClick={handleCompile}
className="px-6 py-2 bg-blue-500 hover:bg-blue-600 text-white font-semibold
rounded-md transition duration-200"
>
Run Code
</button>
<div className="w-full max-w-4xl bg-gray-800 text-gray-200 p-4 mt-6 rounded-lg
shadow-lg">
<h2 className="text-xl font-semibold mb-2">Output:</h2>
<pre className="whitespace-pre-wrap">{output}</pre>
</div>
</div>
);
}
1
1
9
```